SaaS CRM — Developer Documentation

Audience: Engineers working on, extending, or maintaining this codebase. **Stack:** Laravel (PHP), Blade + vanilla JS/Chart.js, MySQL, Spatie `laravel-permission`, `barryvdh/laravel-dompdf`, Razorpay, Meta Graph API (WhatsApp/Instagram/Lead Ads).

---

1. Architecture Overview

This is a **multi-tenant SaaS CRM**. A single codebase and database serve every tenant (business); tenants are isolated by a `tenant_id` column enforced through a shared trait and global query scope, not separate databases or schemas.

1.1 Multi-tenancy — `App\BelongsToTenant`

Almost every model uses this trait:

```
use App\BelongsToTenant;

class Lead extends Model
{
    use BelongsToTenant;
}
```

It does two things via Eloquent's `booted()` hooks:

1. **On create** — auto-fills `tenant_id` from (in order of priority) the authenticated user's `tenant_id`, then `app('tenant_id')` (set by middleware), then the session.
2. **On every query** — adds a global scope filtering `WHERE tenant_id = :current_tenant`, unless the authenticated user `isSuperAdmin()` (superadmin queries are unscoped by design).

! Known gap: `Task` does **not** use `BelongsToTenant`. Every `Task::` query in the codebase must manually filter `->where('tenant_id', $tenantId)` or it will leak other tenants' tasks. This is a deliberate historical exception, not a bug — but it's a trap for new code. Grep for `Task::` before adding new task queries and confirm tenant filtering.

To bypass the scope intentionally (e.g. public/no-login controllers, or superadmin tooling), use:

```
Model::withoutGlobalScope('tenant')->...   // single scope
Model::withoutGlobalScopes()->...          // all scopes
```

1.2 Module Gating (Plan Features)

Every tenant is on a `Plan` (see `app/Models/Plan.php`). A plan has a JSON `features` map of boolean flags (e.g. `manufacturing`, `service`, `subscriptions`, `appointments`, `time_tracking`, `tickets`, `whatsapp`, `reports`, `social_leads`).

A tenant can also carry a **per-tenant override** in `tenant.settings['modules'][$module]` (`true`/`false`), set by superadmin. Resolution order (`Tenant::hasModuleEnabled()`):

1. If an override exists for `$module` → use it (true or false), regardless of plan.
2. Otherwise → fall back to `$tenant->plan->hasFeature($module)`.

Enforce this in routes with the `module:` middleware alias (registered in `bootstrap/app.php`):

```
Route::prefix('/tickets')->middleware('module:tickets')->group(function () { ... });
```

In Blade:

```
@if(auth()->user()->tenant?->hasModuleEnabled('tickets'))
    ...
@endif
```

Adding a new toggleable module — checklist:

1. Add a permission set to `RolesAndPermissionsSeeder.php`.
2. Add the feature key to `PlanController::buildFeatures()` and the `_form.blade.php` `$boolFeatures` array (Plan management UI).
3. Add `'your_module'` to `TenantController::TOGGLEABLE_MODULES` and its label in `moduleLabel()` (generic per-tenant override toggle, reused by every module added since the Subscriptions/Appointments/Time-Tracking/Tickets split — don't duplicate the older `toggleManufacturing`/`toggleService` pattern for new modules).
4. Gate routes with `module:your_module`.
5. Gate sidebar links with `hasModuleEnabled('your_module')`.
6. **Continuity check**: if splitting an existing flag or adding a brand-new one, directly patch any live tenant's `Plan.features` (or tenant override) that should already have access — a new flag defaults to "off" everywhere, which silently revokes access from tenants who were previously relying on a broader flag. This bit us twice during the Service-module buildout.

1.3 Roles & Permissions

Uses **Spatie** **`laravel-permission`** (`HasRoles` trait on `User`). Three built-in roles, seeded by `database/seeders/RolesAndPermissionsSeeder.php`:

- **`superadmin`** — no permission checks anywhere (checked via `User::isSuperAdmin()` / `user_type` column, not via Spatie).
- **`tenant_admin`** — `syncPermissions(Permission::all())`, i.e. every permission that exists.
- **`staff`** — a deliberately minimal default subset (see the seeder for the exact list). **Tenant admins can create additional custom roles** with any permission combination via `Tenant\RoleController` (`resources/views/tenant/roles/*`) — the permission picker there pulls `Permission::all()->groupBy(...)` dynamically, so any newly seeded permission automatically appears with no UI code change needed.

Route-level enforcement:

```
Route::middleware('permission:tickets.view_all|tickets.view_own')->group(...);  // pipe = OR
Route::middleware('permission:tickets.view_all,tickets.view_own')->group(...);  // comma =
AND — easy to get backwards, double-check intent
```

Custom (tenant-created) roles are stored with a `tenant_{id}_` name prefix so `Tenant\RoleController` can distinguish "roles this tenant may edit" from the three global system roles.

Adding a new permission: add the string to the `$permissions` array in `RolesAndPermissionsSeeder.php`, re-run `php artisan db:seed --class=RolesAndPermissionsSeeder --force` (safe — uses `firstOrCreate`, additive only, never removes existing custom-role assignments). Reference it in route middleware exactly as declared — a typo throws `PermissionDoesNotExist` (hard 500), it does not silently deny.

1.4 Billing / Subscription Middleware

`App\Http\Middleware\CheckSubscription` blocks all tenant routes (redirect to an "expired" page, or 402 JSON for API calls) if the tenant's own SaaS subscription (to *this* platform — not to be confused with the tenant's *customers*' service subscriptions, a completely separate feature) is missing or expired. Applied via a `subscription` middleware group.

---

2. Full Module Reference

2.1 Core CRM (always on, not plan-gated)

Module	Controller	Highlights
Leads	`Tenant\LeadController`	Status/bulk-status update, bulk assign/delete, lead→deal conversion, call-log entries, duplicate check, saved views. Import/export via `LeadImportController`. Fed by lead-source webhooks (§2.4).

Module	Controller	Highlights
Contacts	`Tenant\ContactController`	CRUD + attachments, employee sub-records (`ContactEmployee`) with their own attachments, `searchCustomers` autocomplete, `customerReport`. Import/export via `ContactImportController`; `DuplicateController` merges duplicates.
Deals	`Tenant\DealController`	Pipeline stage updates, `markWon` / `markLost`, CSV export, kanban/funnel `pipelineAnalytics`.
Tasks	`Tenant\TaskController`	Checklist items, comments, attachments, watchers, dependencies, bulk actions, saved filters. Reusable `TaskTemplate`s. **See the tenant-scoping caveat in §1.1.**
Follow-ups	`Tenant\FollowupController`	CRUD, `markDone` / `markMissed`, attachments.
Quotations	`Tenant\QuotationController`	PDF generation, `send()` (email/WhatsApp), `convertToInvoice`, versioning (`newVersion`), configurable terms templates. **Public e-sign portal** at `/quote/{token}` (`Public\QuotationController`) — customer views/accepts/rejects with no login.
Invoices	`Tenant\InvoiceController`, `InvoicePayment`	PDF (branding configurable via `InvoicePdfSettingController`), `send()`, `recordPayment` (partial/multiple payments supported). No public-facing page exists for invoices today (unlike quotations).

2.2 Manufacturing Suite (`module:manufacturing` for Work Orders; the rest are always visible)

Full flow: **Purchase Request → Approval → Purchase Order (optionally via Vendor Quote comparison) → Receive (stock increases) → Work Order consumes stock → produces finished goods (stock increases) → Product Batches tracked throughout.**

Model / Controller	Role
`Product` / `ProductController`	Catalog, low-stock listing (`Product::scopeLowStock()`), per-product batch listing, AJAX search.
`ProductBatch`	Lot/batch tracking, used by receiving and Work Order completion.
`Vendor` / `VendorController`, `VendorQuote` / `VendorQuoteService`	Vendor master + RFQ-style quote comparison feeding into POs.

Model / Controller	Role
`PurchaseRequest` / `PurchaseRequestService`	`create → approve()/reject()`.
`PurchaseOrder` / `PurchaseOrderService`	Created from a PR or standalone; PDF + `send()`; `receive()` updates stock and can create `ProductBatch` rows.
`WorkOrder` / `WorkOrderService`	Production job against a `Product`. `create → start() → complete()` (consumes materials, produces stock) or `cancel()`; `updateCosts()` for labor/machine/material cost breakdown.
`StockService`	Shared inventory-quantity ledger used by both PO receiving and Work Order completion — not a UI module, the single source of truth for stock math.

2.3 Service Business Suite

Independently toggleable modules (`module:subscriptions`, `module:appointments`, `module:time_tracking`, `module:tickets`) plus the base `module:service` catalog. Built out across this project's service-business initiative; see §4 for the automation layer behind these.

Module	Summary
Service Catalog	`Service` model — `rate`, `tax_percent`, `unit`, `billing_cycle`, `duration_value/unit`, `total_quantity`, `is_package` (a Package *is* a Service with `is_package=true` plus a `service_package_items` pivot of component services — not a separate model).
Subscriptions	`ServiceSubscription` — per-customer tracking of one active service instance with `expires_at`, usage quantity (`total_quantity` / `used_quantity`), status (`active` / `expired` / `cancelled`). Customer-facing Email/WhatsApp expiry reminders with tenant-editable channel toggle + message template (`SubscriptionReminderService`, `Tenant\ServiceSubscriptionController::updatePreferences` / `sendTestEmail`). Bulk renew/cancel. History log (`SubscriptionReminderLog`).
Appointments / Booking	`Appointment` — capacity-based slot booking (not per-staff), configurable business hours/slot length/advance-booking window (`Tenant::bookingSettings()`). **Public booking page** at `/book/{token}` incl. customer self-cancel. Auto customer reminder 24h before (`appointments:remind-upcoming` cron) and auto no-show marking 2h after end time if status was never updated (`appointments:mark-no-show` cron).
Time Tracking	`TimeEntry` — start/stop timer or manual entry, `is_billable` / `is_invoiced` flags, `hourly_rate`, `duration_minutes` (always store as `(int) round(...)` — `Carbon::diffInMinutes()` returns a float and the column is an unsigned integer). `convertToInvoice` bundles selected entries into an invoice.

Module	Summary
Tickets / Helpdesk	`Ticket`, `TicketReply` (with `is_internal_note` filtered out of customer-facing views), `TicketAttachment`. Human-readable `ticket_number` (`TKT-YYYYMMDD-NNNN`, `Ticket::generateNumber()`). **Public submission form** at `/support/{token}` and a per-ticket tracking page. Automatic Email/WhatsApp confirmation on creation, tenant-configurable channel + template (`TicketNotificationService`). SLA breach detection by priority (`tickets:check-sla` — urgent 1h / high 4h / medium 24h / low 48h, "no staff reply yet").

2.4 Lead Source Integrations

Platforms: **Meta Lead Ads**, **IndiaMART**, **JustDial**, **TradeIndia**, **Sulekha** (`TenantIntegration::PLATFORMS`). Superadmin controls a per-tenant allow-list (`SuperAdmin\LeadIntegrationController`); tenants then configure credentials themselves (`Tenant\LeadIntegrationController` — setup/save/regenerateToken/syncNow/testConnection). Each platform has its own credential shape. Generic webhook ingestion also exists (`GenericLeadWebhookService`, `/webhook/leads/{token}`). Full docs: `docs/lead-integrations/`.

2.5 Communication

- **Email** (`Tenant\EmailController`, `EmailService`, `EmailSetting`, `EmailTemplate`, `EmailLog`) — send/bulk-send, templates, logs, **tenant-supplied custom SMTP** (tenants can connect their own mail server instead of the platform default).
- **WhatsApp** (`Tenant\WhatsappController`) — same shape as Email; gated by the plan's `whatsapp` feature.
- **WhatsApp Chatbot** (`Tenant\WhatsappChatbotController`, `WhatsappChatbotFlow/Session`) — separate sub-feature: Meta WhatsApp Business API OAuth (incl. QR-code connect), automated conversational flow builder.
- **WhatsApp Team Inbox** (`module:whatsapp_inbox`, registered in `config/modules.php`, tier `pro`) — shared inbox over `whatsapp_logs`; the tracker, data model and rollout notes live in `docs/whatsapp-team-inbox.md`. Pieces:
  - Inbox (`Tenant\WhatsappInboxController`, `WhatsappConversation`, permission `whatsapp.inbox`; reply also needs `whatsapp.send`). Messages are not copied: `whatsapp_logs` rows with `status='received'` are the customer, everything else is us. `WhatsappInboundService::handle()` → `WhatsappConversation::touchFor()` feeds it from both the direct-Meta and the gateway webhook. UI is one page driven by JSON endpoints (`show`, `poll`, `reply`, `status`, `assign`, `notes`) — no page reloads.
  - Team features: assignment + round-robin auto-assign (`whatsapp_settings.inbox_auto_assign*`), `whatsapp_quick_replies`, `whatsapp_conversation_notes`; permissions `whatsapp.assign`,

`whatsapp.view_all` (without it a user sees only own + unassigned chats, enforced in `scoped()`), `whatsapp.quick_replies`.

- Reports (`Tenant\WhatsappInsightsController`, read-only): `whatsapp/performance` (permission `whatsapp.view_reports`; first-reply time from log order per phone, human replies only) and `whatsapp/campaigns` (permission `whatsapp.view_logs`; a campaign = `bulk_id`). Bulk Send got front-end status/source filters.
- Rollout: migrations `2026_10_13_100000`, `2026_10_14_100000`, `2026_10_15_100000` create the tables/columns and grant the new permissions to the `tenant_admin` / `staff` roles; run `php artisan migrate`. Plans: `PlanSeeder` bundles the module from Pro; superadmin can also tick it in the plan editor or toggle it per tenant.
- Step 3b: broadcast opt-out (`whatsapp_opt_outs`, `WhatsappOptOut`, STOP/START in `WhatsappInboundService::handleOptOut`), automatic Lead from a first inbound message (`autoCreateLead`, module-gated), scheduled bulk sends (`whatsapp_scheduled_broadcasts`, command `whatsapp:send-scheduled` every minute, shared sender `Services\WhatsappBulkSender`). Migration `2026_10_16_100000`. Needs the scheduler cron.
- Known gap: delivery status only comes from the gateway webhook (which folds *read* into *delivered*); the direct-Meta webhook ignores status events.

- **Instagram** (`Tenant\InstagramController`, tenant_admin only) — OAuth connect, comment/DM auto-reply automations, its own chatbot flow builder. This is the feature behind the plan's `social_leads` flag. Docs: `docs/instagram-automation/`.
- **Slack** (`Tenant\SlackController`, tenant_admin only) — incoming-webhook-based internal alerts (new lead, deal won, etc.).

2.6 Reports (`Tenant\ReportController`, `permission:reports.view_basic|reports.view_all`)

Overview · Deals · Deal-Quotations · Revenue · Staff · Subscriptions · Appointments · Tickets · Time-Tracking. (There is intentionally no separate "Leads" or "Products" report tab today — see Roadmap.)

2.7 HR & Other Notable Features

- **Calendar** (`Tenant\CalendarController`) — unified drag-to-reschedule view aggregating tasks, follow-ups, and appointments.
- **Attendance & Screenshots** (`Tenant\AttendanceController`, `Tenant\ScreenshotController`) — clock-in/out, manual/bulk entry, periodic desktop screenshot capture — distinct from the Time Tracking task-timer module.
- **Duplicate Detection** (`Tenant\DuplicateController`, `DuplicateMatcher`) — find & merge duplicate Leads/Contacts.
- **Global Search** (`Tenant\SearchController`).

- **Audit Logs** (`HasAuditLog` trait, `AuditLog` model) — tenant_admin-visible change history.
- **Notifications** (`Tenant\NotificationController`, `config/notifications.php`) — in-app bell + per-channel preferences; Firebase-backed push (`Firebase\NotificationService`, `DeviceToken`) for mobile. Adding a new notification type = add one entry to `config/notifications.php['types']`; the rest of the pipeline (preferences UI, dispatch) picks it up automatically.
- **Custom Fields** (`Tenant\CustomFieldController`) — per-tenant, per-module field builder. Plus a **Global Field Template library** (`GlobalFieldTemplateSeeder`, `Tenant\TenantFieldController`) tenant_admins can assign fields from, alongside fully custom ones.
- **"AI Automation"** (`Tenant\AutomationController`, `WorkflowTemplate`/`WorkflowRequest`) — **not** an in-app automation builder. It's a request-a-custom-automation marketplace: tenants browse a superadmin-curated catalog, then submit a request that lands in the superadmin's fulfillment inbox.
- **Developer/Public REST API** (`routes/api.php`, `Api\Tenant\*`) — versioned `v1` API (Contacts/Leads/Deals/Quotations/Tasks) via tenant-issued API keys (`Tenant\ApiKeyController`, `AuthenticateWithApiKey`). Separate Sanctum auth (`Api\Auth\AuthController`) backs a mobile app.
- **Outgoing Webhooks** (`Tenant\WebhookController`, `TenantWebhook`) — tenant-configurable outbound webhooks (e.g. for n8n/Zapier-style automation), mirrored read-only on the superadmin side per tenant.
- **Coupons** (`SuperAdmin\CouponController`) — discount codes applied at the tenant's own subscription checkout (`Tenant\SubscriptionController::applyCoupon`) — not to be confused with the Service Subscriptions product feature.
- **Billing/Checkout** (`Tenant\SubscriptionController`, `RazorpayService`) — the CRM's own SaaS plan picker/checkout/cancel flow.

2.8 Superadmin Panel

Tenants (list, activate/suspend, module overrides) · Plans (full CRUD + feature/pricing config) · Coupons · Lead-integration access control · Roles & Permissions (edit `tenant_admin`'s base set, master permission list) · Error Logs (app exceptions, resolve/delete) · Workflow Templates (automation catalog) · Workflow Requests (tenant automation-request inbox) · Tenant Webhooks (manage on a tenant's behalf) · Platform Settings (global Meta App credentials shared by every tenant's Instagram/WhatsApp/Lead-Ads OAuth).

---

3. Public (No-Login) Pages

Three flows expose customer-facing pages outside all `auth`/`tenant` middleware, using an unguessable per-record token instead of a session:

Flow	Entry route	Controller
Quotation e-sign	`/quote/{token}`	`Public\QuotationController`
Appointment booking + self-cancel	`/book/{token}`	`Public\AppointmentController`
Support ticket submission + tracking	`/support/{token}`	`Public\TicketController`

Pattern: a **permanent per-tenant token** (`settings['booking_token']` / `settings['support_token']`, mirrored via `Tenant::ensureBookingToken()` / `ensureSupportToken()`) is the entry point for *new* submissions; a **per-record token** (`Str::random(40–48)`, `Model::ensurePublicToken()`) lets the customer track/manage that one specific record afterward. Lookups use `Model::withoutGlobalScope('tenant')` since there's no authenticated tenant context.

---

4. Notification & Reminder Automation

4.1 The pattern

Customer-facing (a `Contact`, not a `User`) transactional messages **don't** go through `NotificationService` (it's hard-wired to a `User` recipient). Instead, each area has its own small service that:

1. Resolves the tenant's channel preference (`Tenant::wants*(string $channel): bool`, defaulting ON unless the tenant has explicitly opted out) and message template (`Tenant::*Template(string $key): ?string`, `null` = use the built-in default constant).
2. Substitutes `{{placeholder}}` tokens (`strtr($template, $pairs)`).
3. Sends via `EmailService::send()` (with an `Illuminate\Support\Facades\Mail::send([],[],...)` fallback if that returns false) and/or `WhatsappChatbotService::forTenant($id)->sendMessage()` (only if `WhatsappSetting::forTenant($id)->is_connected`).
4. Wraps everything in `try/catch` — a notification failure must never break the primary action (ticket creation, booking, reminder send).

Existing implementations of this pattern: `SubscriptionReminderService`, `AppointmentReminderService`, `TicketNotificationService`. **Copy one of these, don't build a fourth variant from scratch** — the `email()` / `whatsapp()` private-method boilerplate is identical across all three by design.

Blade gotcha: literal `{{placeholder}}` text inside a Blade `{{ }}` echo breaks the compiler — use `@{{placeholder}}` to render it literally (e.g. in a "available placeholders" hint under a template textarea).

4.2 Scheduled Commands (`routes/console.php`)

Command	Cadence	Purpose

`leads:check-sla`	every 15 min	Flag "new" leads with no contact within SLA window.
`followups:remind`	every 5 min	Due/overdue follow-up reminders.
`invoices:remind-payments`	daily	Payment due/overdue reminders.
`tasks:remind`	daily 09:00	Task due-date reminders.
`products:check-batch-expiry`	daily 08:00	Product batch expiry alerts.
`subscriptions:remind-expiry`	daily 08:30	Customer subscription expiry reminders (respects per-tenant days-before setting).
`appointments:remind-upcoming`	hourly	Customer reminder for appointments starting within 24h (`--hours=` configurable), guarded by `reminder_sent_at` against duplicates.
`appointments:mark-no-show`	hourly	Auto-marks `booked` / `confirmed` appointments as `no_show` once 2h (`--hours=`) past `ends_at` with no staff status update.
`tickets:check-sla`	every 15 min	Flags open/in-progress tickets with zero staff replies past their priority's SLA window; notifies assignee or all tenant_admins.
`App\Jobs\SyncIndiaMartLeadsJob`	every 30 min	Pulls new IndiaMART leads.

Production note: these only fire if something is actually invoking `php artisan schedule:run` every minute (system cron / Task Scheduler / Herd's built-in scheduler for local dev). Confirm this is wired up on any new deployment target — the code being correct doesn't guarantee the trigger exists.

4.3 Notification Types Registry

`config/notifications.php['types']` is the single source of truth for every **staff-facing** (`User` recipient) notification: label, icon, color, group, default channels, message template. Adding a type here automatically wires it into the in-app bell, the preferences UI, and email dispatch — no other file needs to change. Dispatch via the static helper:

```
NotificationService::notify('ticket.sla_breach', $recipient, ['subject' => ..., 'priority' => ...], $by = null, $url, $notifiableModel);
```

---

5. Testing Discipline Used In This Codebase

There is no formal test suite covering most of the modules above. The working pattern established during development (and worth continuing) is: **standalone PHP scripts that bootstrap Laravel manually**, run outside the request lifecycle, exercise real code paths against the real (dev) database, and clean up after themselves:

```
 require 'vendor/autoload.php';
$app = require 'bootstrap/app.php';
$app->make(Illuminate\Contracts\Console\Kernel::class)->bootstrap();

Illuminate\Support\Facades\Auth::login($someUser);   // needed for tenant-scoped queries
$controller = new App\Http\Controllers\Web\Tenant\SomeController();
$request = Illuminate\Http\Request::create('/path', 'POST', [...]);
$request->setUserResolver(fn() => $someUser);
$result = $controller->someMethod($request);
// assert via echo/PASS-FAIL, then delete every row you created
```

This caught several real bugs that pure code review missed, including: a pivot-table sync silently dropping a required `tenant_id` column, a float being stored into an unsigned-integer column, a `$fillable` omission causing a mass-assigned field to be silently dropped, a `@json()` Blade directive choking on a nested arrow-function expression, and a JS `//` comment containing literal `@if`/`@endif` text that Blade's compiler parsed as real directives. **Always render the actual Blade view (`$view->render()`), not just call the controller method** — several of the above only surface at render time.

PHP CLI in this dev environment: `C:\Users\admin\.config\herd\bin\php84\php.exe` (not on `PATH`).

---

6. Known Limitations & Future Roadmap

Honest, current-as-of-this-writing list of gaps and deliberate scope cuts — useful both as a backlog and as an answer to "is X supported":

- **Subscription renewal is manual.** The expiry reminder fires automatically, but generating the renewal invoice still requires a staff click ("Renew" / "Bulk Renew"). Fully automatic invoice generation on expiry was deliberately deferred — it's a financial action with real risk if a customer intends to cancel rather than renew, so it needs a human in the loop for now. If this changes, it should generate a **draft** invoice, never an auto-sent/auto-charged one.
- **Appointment reminders aren't tenant-customizable yet.** Unlike Subscription and Ticket confirmations (which have a full channel-toggle + editable-template settings panel), the appointment reminder message is a fixed template. Bringing it up to parity is a contained, well-

understood piece of work (copy the `TicketNotificationService` / `Tenant\TicketController::updatePreferences` pattern).

- **No dedicated "Leads" or "Products" report tab.** Both are core, high-traffic modules; a report tab for each would be a natural next addition, mirroring the shape of the existing Subscriptions/Appointments/Tickets reports.
- **No SMS channel.** Only Email, WhatsApp, in-app, push (mobile), and Slack exist today.
- **No persistent customer login/portal.** Customer-facing access is entirely via one-off unguessable tokens (quotation e-sign, appointment tracking, ticket tracking) rather than a real account a customer logs into to see all their history in one place. Would be a significant addition, not a small one.
- **Appointments are capacity-based, not per-staff.** A booking consumes one slot of a shared capacity; there's no "book with a specific technician/stylist" concept. Deliberate simplification — revisit if a service vertical needs per-staff scheduling.
- **No-show auto-marking is heuristic.** Any `booked` / `confirmed` appointment left untouched 2 hours past its end time gets marked `no_show` automatically. This can't distinguish "genuinely didn't show up" from "showed up but staff forgot to click Complete" — it's a safety net, not a certainty.
- **Default `staff` role permissions are intentionally minimal.** Tenant admins are expected to build custom roles (fully supported in-app) for anything beyond the seeded default. The default set was last expanded to give staff basic day-to-day capability in the four newest service modules (create a subscription; edit/cancel their own appointments; edit their own time entries; progress a ticket's status/priority/assignment) — review it again whenever a new module ships.
- **No invoice payment link for customers** (unlike quotations, which have a public e-sign page). Invoices currently have no public-facing page at all.
- **Housekeeping**: `app/BelongsToTenant copy.php` and `routes/web copy.php` are stray backup files sitting in the repo — confirm nothing references them and delete.
- **Dashboard trend badges** currently cover Leads / Revenue / Deals-created / Tasks-completed month-over-month. Manufacturing (stock/purchase-request alerts) and the four service modules (Subscriptions/Appointments/Time Tracking/Tickets alert banners) exist as *alert banners*, not full tred cards — extending the KPI-card row itself to include manufacturingn/service metrics is a natural next step once a tenant's mix of active modules is known.
- **"AI Automation" is a request marketplace, not a builder.** Tenants cannot self-serve build a workflow today — every automation is fulfilled manually by the platform team after a request comes in. Framing this correctly to prospects/users matters (see the Seller and User guides) so it isn't mistaken for a Zapier-style self-service tool.